

Python Programming for non-Programmers



UNIVERSITÄT
DES
SAARLANDES

Saarland University

Summer Term 2026

M.Sc. Patrick Volkmann

Software Project: Peak Pulse Fitness

Issue:	01.06.2026, 08:00 pm
Submission code and report:	14.08.2026, 11:59 pm

1. Introduction & Business Case

Your group is developing a demo version of a custom web management platform for **Peak Pulse Fitness** to present to their management. As a modern business, they have already implemented systems to collect data in their own database, which they now want to use to analyze customer profiles, optimize product inventories, rank department performances, and run automated marketing or recommendation tasks.

For this project, you will use the pre-populated SQLite database and raw CSV data provided in this folder. Your goal is to build a unified web platform that integrates this data, provides full administrative CRUD functionality, and implements two distinct individual analytics and logic tracks.

Terminology Mapping

- **Primary Entity (Table A):** Gym Member (`Customer\_data`)
- **Transaction/Interaction (Table B):** Session (`Order\_data`)
- **Employee / Resource (Table C):** Staff Member (`Employees\_data`)

2. Relational Database Schema

Your database (`5\_gym\_fitness.db`) contains the following pre-compiled tables:

1. `Customer\_data` (Gym Member)

Contains registered gym members.

- **`CustomerID`:** Unique ID (Primary Key).
- **`FirstName`:** First name.
- **`LastName`:** Last name.

- **`Age`**: Age of the participant.
- **`BirthDate`**: Date of birth (formatted as a string `YYYY-MM-DD`).
- **`Country`**: Country of origin (Germany, Austria, or Switzerland).

2. **`Order\_data` (Session)**

Tracks individual transactions/bookings.

- **`OrderID`**: Unique ID of the transaction (Primary Key).
- **`CustomerID`**: The ID of the gym member who placed it (Foreign Key referencing `Customer\_data`).
- **`EmployeeID`**: The ID of the staff member who processed the transaction (Foreign Key referencing `Employees\_data`).
- **`Description`**: The specific `Training / Pass Class` sold/booked.
- **`Price`**: The price per item.
- **`Amount`**: The quantity/units.
- **`Date`**: Date of transaction (formatted as a string `YYYY-MM-DD`).

3. **`Employees\_data` (Staff)**

Tracks the internal company roster.

- **`EmployeeID`**: Unique ID (Primary Key).
- **`FirstName`**: First name.
- **`LastName`**: Last name.
- **`Email`**: Employee contact email.
- **`Age`**: Age of the employee.
- **`Country`**: Region or city of the employee.
- **`PhoneNumber`**: Contact phone number.
- **`HireDate`**: Date of hiring (formatted as `YYYY-MM-DD`).
- **`JobID`**: Job title / role (Director, Store Manager, or Sales Associate).
- **`Salary`**: Monthly salary in EUR.
- **`ManagerID`**: Self-referencing Foreign Key to `EmployeeID` (hierarchical supervisor).
- **`DepartmentID`**: The department or cost-center ID.

3. Project Tasks & Deliverables

Part I: Core Tasks (Joint Group Work - 60 Points total)

Both group members are collectively responsible for the foundation of the application (Task 0, Task 1, Task 2). These core tasks are graded within the 60-point Group Score as follows:

- **Task 0: Code Quality & Documentation**

Ensure all code is cleanly structured, commented, and documented. Use meaningful variable names and helper functions so that your partner and evaluators can easily follow your logic. *(Assessed as part of the overall 20-point 'Quality of Solution' and 20-point 'Project Report' criteria).*

- **Task 1: Database Integration**

Set up and integrate the SQLite database into your Flask application using SQLAlchemy. Ensure correct relationships (One-to-Many, Foreign Keys) are maintained in your `models.py` implementation. *(Evaluated as 10 points under the 'Core Code Implementation' criteria).*

- **Task 2: CRUD Data Management UI**

Build a user-friendly web interface allowing an administrative user to manually manage your primary entity (`Customer\_data`) and transaction (`Order\_data`) records. The UI must support full CRUD:

Create: Insert new records through web forms (using WTForms/Flask-WTF).

Read: Display existing records in a clear tabular format.

Update: Edit existing fields safely.

Delete: Remove outdated or incorrect entries. *(Evaluated as 10 points under the 'Core Code Implementation' criteria).*

Part II: Symmetrical Individual Extensions (80 Individual Points total)

To split the 80 individual points (60 points for code extensions + 20 points for the oral video presentation) perfectly and fairly, both students face the exact same technical challenges but focus on different business entities. Choose one of the two roles below:

Student 1: Customer & Revenue Focus (60 Individual Code Points)

- Task 1.A: Customer Demographics Dashboard (20 Points)

Perform business intelligence analyses on customer data using Pandas and plot visualizations using Matplotlib.

- **Plot 1: Geographic Distribution** - Visualize customer geographic origins (e.g. bar or pie chart).
- **Plot 2: Age Segmentation** - Cross-analyze customer age groups against spending habits to show how age correlates with order characteristics (e.g. scatter plot or grouped bar chart).

Hint: To display charts on the website without blocking the Flask server, save the plot as a static image file (e.g., in your static folder) and link to it in your HTML template.

- Task 1.B: Customer Loyalty & Spending Leaderboards (20 Points)

Use SQL / SQLAlchemy aggregations to find and display real-time rankings:

- **Leaderboard A (Loyalty):** Find and list the top customers who placed the highest number of transactions.
- **Leaderboard B (Financial):** Find and list the top customers who generated the highest total revenue/spending.

- Task 1.C: Automated Customer Campaigns (20 Points)

Implement trigger-based campaign logic in Python:

- **Birthday Campaign:** Query the database to find customers whose birthday month matches the current month.
- **Inactive Accounts:** Identify customers who have not placed a transaction/booking in the past 6 months.

Hint: Dates in the database are stored as strings (YYYY-MM-DD). You can either extract the month using Python string slicing (e.g., `birthdate_str[5:7]`) or convert it to a datetime object for comparison.

Student 2: Staff & Product/Service Focus (60 Individual Code Points)

- Task 2.A: Staff & Product Performance Dashboard (20 Points)

Perform business intelligence analyses on inventory and staff data using Pandas and plot visualizations using Matplotlib.

- **Plot 1: Product Popularity** - Visualize the top 10 most popular product models or services (e.g. horizontal bar chart).
- **Plot 2: Staff Distribution** - Cross-analyze staff members to show department sizes or salary bands (e.g. pie chart or histogram).

Hint: Just like Student 1, save your chart as a static image file in your Flask static folder and render it in your HTML template.

- **Task 2.B: Staff Performance & Department Revenue Leaderboards (20 Points)**

Use SQL / SQLAlchemy aggregations to find and display real-time rankings:

- **Leaderboard A (Staff):** Find and list the top employees based on the number of transactions they processed.
- **Leaderboard B (Departments):** Find and list the top departments/roles based on the total revenue they generated.

- **Task 2.C: Custom Recommendation Engine (20 Points)**

Implement a rule-based algorithm to encourage repeat business:

- **Product Recommendations:** Analyze customer transaction histories to recommend additional items or services (e.g. if a customer frequently purchases item A, automatically recommend related item B).
- **Marketing Support:** Identify low-performing products/services that need marketing support.

Hint: You are not expected to use complex machine learning libraries here. A simple rule-based approach (e.g., using a Python dictionary to track which items are frequently bought together) is completely fine.

4. Oral Presentation & Defense (Video Colloquium) - 20 Points

To demonstrate individual understanding and defend your code, each student must record and submit a **5-7 minute individual presentation video** (combined into one 10-15 minute group submission).

Required Video Structure:

1. **Contribution Matrix (Start):** Clearly state which student took on the Customer & Revenue Focus (Student 1) and who took on the Staff & Product Focus (Student 2).

2. Individual Browser Demo (1.5 minutes per student): Perform a live demonstration of your custom extensions (dashboard, rankings, and logic triggers) in action in your web browser.

3. Code Walkthrough (3 minutes per student): Walk through your custom Python code in the IDE (Flask routes, Pandas scripts, SQLAlchemy queries, and rule logic). Explain **why** you wrote it this way, demonstrating that you fully understand the mechanics.

4. Bug & Reflection Walkthrough (1.5 minutes per student): Highlight specific programming bugs, logic errors, or AI-generated hurdles you encountered, and explain the logical steps you took to debug and solve them. Also explain which AI tools you used and where they failed or generated buggy code.

5. Written Project Report Guidelines

Each group must submit a comprehensive, **maximum 15-page project report** explaining their system's design and code.

Required Report Structure:

1. Title Page & Executive Summary: Overview of your chosen business case and system name.

2. System Architecture & Database Design: Description of the database schema (Table A, B, and C) and how they relate via foreign keys and SQLAlchemy models.

3. Core Functionality (Part I): Documentation of the database connection and the CRUD interface (how data is validated and managed).

4. Individual Extension Highlights (Part II):

Student 1 Section: Detailed explanation of the customer demographics plots, loyalty algorithms, and string-to-date campaign triggers.

Student 2 Section: Detailed explanation of the popular inventory plots, department revenue queries, and item pairing recommendations.

5. Generative AI Reflection (Mandatory): Identify all Generative AI tools used (e.g., ChatGPT, Claude) during development. You must document:

What was generated (e.g., boilerplate, SQL syntax).

Where AI tools failed or generated buggy code.

How you critically reviewed, debugged, and edited the code to make it work.

6. Installation & Setup Instructions: Provide clear, step-by-step instructions so your TAs can run your application locally.

Formatting Guidelines:

- **Font:** Times New Roman or Arial (Size 12)
- **Line spacing:** 1.5
- **Margins:** 2.5 cm on all sides

6. Submission Guidelines & Technical Requirements

To ensure a smooth grading process and avoid technical upload issues on Moodle, all groups must adhere to the following submission protocol:

1. The Moodle Package (One Submission per Group)

Only **one member** of each group should upload the final project folder as a single `.zip` file. The archive must contain exactly one file and one folder:

- **Project_Report.pdf:** Your completed 15-page project report.
- **project_code/:** Your clean, complete Flask application directory.

CRITICAL: Exclude Heavy Folders! Before zipping your `project_code` folder, you **must delete** your virtual environment folder (e.g., `.venv`, `env`, or `venv`), your `.git` history, and any `__pycache__` folders. Zipping these folders will make your file too large for Moodle and will result in a failed upload.

2. Video Upload Requirements

Because high-quality 10-15 minute video files are typically too large for standard Moodle upload limits, we will provide a dedicated folder/link on the Moodle course page.

- **HIZ-Cloud Upload Link:** Upload your MP4 or MOV video presentation directly to the dedicated HIZ-Cloud folder/link provided on Moodle.
- **Optional Confirmation:** If you want to make absolutely sure we will find your video, you can optionally include the HIZ-Cloud upload link you used on the **Cover Page of your PDF report** or in a text file within your submission.

7. Grading & Point Distribution (140-Point Map)

Your final grade is composed of a **60-point Group Score** and an **80-point Individual Score**:

1. Group Performance (Maximum 60 Points)

- **Project Report (20 Points):** Evaluated on structural clarity, database schema description, code documentation, and the mandatory AI reflection chapter.
- **Quality of Solution (20 Points):** Evaluated on the overall UX, navigation, styling, visual look-and-feel, and bug-free operation of the application.
- **Core Code Implementation (20 Points):**

Task 1: System Architecture (10 Points) - Relational SQLite setup, Flask configuration, and SQLAlchemy tables.

Task 2: CRUD Management UI (10 Points) - Data insertion, retrieval tables, update forms, and delete logic.

2. Individual Performance (Maximum 80 Points per student)

- **Individual Code Extensions (60 Points):**

Task A: Data Dashboard & Visualization (20 Points) - 2 custom Matplotlib charts built via Pandas aggregations.

Task B: Performance & Revenue Leaderboards (20 Points) - 2 active SQLite ranking tables using SQLAlchemy aggregations.

Task C: Automated Logic Triggers (20 Points) - Customized business automation code (date campaign vs. recommendation rule).

- **Oral Video Presentation (20 Points):** Evaluated on the clarity of your code walkthrough, demo effectiveness, and critical bug reflection.